

Reviewer Pack — Quantum Entanglement in Boolean Functions

Entry a2b0e2b6bd54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 16:25:25 UTC

Quantum Entanglement in Boolean Functions

Entry ID: a2b0e2b6bd54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 16:25:25 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

The entanglement entropy of the completely mixed state on a boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ is bounded by $O(n \log n)$ and is equal to the number of qubits needed for a quantum circuit that computes f in polynomial time.

Rationale (proposer's reasoning):

Quantum entanglement measures the correlation between qubits in a quantum state. By linking it to boolean functions, we might uncover new structural properties in circuit complexity. If the conjecture holds, it suggests that quantum algorithms can efficiently extract information about the complexity of boolean functions, potentially providing insights into P vs NP.

Taxonomy category: quantum_information_theory_boolean_circuit_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dcee43037880aa98

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Entanglement entropy of a boolean function is bounded by $O(n \log n)$ and equals the number of qubits for polynomial-time quantum circuit computation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum entanglement" AND "Boolean function" AND complexity - "entanglement entropy" IN BOOLEAN FUNCTIONS AND quantum information theory - "polynomial time quantum circuit" AND Boolean circuit complexity AND entanglement

Top relevant hits considered: - [<http://arxiv.org/abs/quant-ph/0510050v2>] Constant communication complexity protocols for multiparty accumulative boolean functions - [<http://arxiv.org/abs/0906.2946v1>] Bi-partite entanglement entropy in massive 1+1-dimensional quantum field theories - [<http://arxiv.org/abs/2407.10000v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/hep-th/9707234v2>] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [<http://arxiv.org/abs/1906.00324v1>] Ubiquitous Complexity of Entanglement Spectra - [<http://arxiv.org/abs/2410.13927v1>] Exploiting recursive structures for the design of novel quantum primitives - [<http://arxiv.org/abs/2303.01877v4>] Quantum Merlin-Arthur proof systems for synthesizing quantum states

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_entanglement_entropy(boolean_function):
        n = int(math.log2(len(boolean_function)))
        # Simplified computation of entanglement entropy (not actual quantum computation)
        return n * math.log(n, 2)

    def compute_qubits_needed(boolean_function):
        n = int(math.log2(len(boolean_function)))
        # Simplified computation of qubits needed (not actual quantum circuit simulation)
        return n

    metric_name = "Spearman rank correlation coefficient"
    instances_tested = 0
    n_max = 0
    entanglement_entropies = []
    qubit_counts = []
```

```

for n in [5, 10, 15, 20, 30, 40]:
    boolean_function = generate_boolean_function(n)
    entanglement_entropy = compute_entanglement_entropy(boolean_function)
    qubit_count = compute_qubits_needed(boolean_function)

    entanglement_entropies.append(entanglement_entropy)
    qubit_counts.append(qubit_count)

    instances_tested += 1
    n_max = max(n_max, n)

if instances_tested < 30:
    return {
        "metric_name": metric_name,
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

def spearman_rank_correlation(x, y):
    x_ranks = {x[i]: i for i in range(len(x))}
    y_ranks = {y[i]: i for i in range(len(y))}
    n = len(x)
    sum_d_squared = sum((x_ranks[x[i]] - y_ranks[y[i]]) ** 2 for i in range(n))
    return 1 - (6 * sum_d_squared) / (n * (n**2 - 1))

correlation_coefficient = spearman_rank_correlation(entanglement_entropies, qubit_counts)

conjecture_holds = abs(correlation_coefficient - n_max) < 0.5
counterexample = "" if conjecture_holds else f"correlation={correlation_coefficient}, n_max={n_max}"

return {
    "metric_name": metric_name,
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    supported_trials = [r for r in results if r["conjecture_holds"]]
    support_fraction = len(supported_trials) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in supported_trials)/len(supported_trials)}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in supported_trials)/len(supported_trials)}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"correlation_mismatch\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15062
2	preregistration	ollama_remote	glm4:latest	0	0	9311
3	novelty	ollama_remote	glm4:latest	0	0	20724
4	novelty	ollama_remote	glm4:latest	0	0	9595
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17130
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42336
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	50282
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33530
9	judge	ollama_remote	glm4:latest	0	0	37183

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 235153 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/a2b0e2b6bd54.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a2b0e2b6bd54.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a2b0e2b6bd54.tar.gz` (if generated)